

Enhancing Alignment of Language Models through Safe-Data Interleaving

data-to-paper

March 31, 2026

Abstract

The alignment of large language models with human ethical values and safety standards is crucial due to increasing AI integration into daily applications, yet these models remain prone to misalignment issues. This study addresses the gap in knowledge surrounding effective strategies for correcting such misalignments by evaluating safe-data interleaving as a corrective mechanism. Utilizing a dataset where varying quantities of safe examples are interleaved with unsafe data, we explore interleaving ratios ranging from 2 to 100 safe examples per unsafe example. Through Kruskal-Wallis tests and Multivariate Analysis of Variance (MANOVA), our findings indicate that, while there is no significant difference in the total number of training examples or average content length across the different interleaving strategies, there are intricate variations in content structure and interaction dynamics that suggest complex model realignment processes. The analysis reveals potential shortcomings in dataset size and methodological assumptions that affect result interpretation. Despite these limitations, our research provides a foundation for the potential optimization of alignment strategies, paving the way for future investigations into refining safe-data interleaving for improved model compliance with desired norms.

Introduction

The alignment of artificial intelligence (AI), particularly large language models, with human ethical values and safety standards has become increasingly crucial as their integration into various applications continues to expand. Despite ongoing efforts to improve alignment methodologies, language models frequently exhibit misalignments that pose ethical and safety-related

challenges [1]. Addressing these misalignments is essential not only for enhancing the performance and reliability of AI systems but also for ensuring compliance with societal norms, especially within sensitive domains [2]. As a result, understanding and implementing robust alignment strategies such as safe-data interleaving is a significant area of inquiry [3].

Previous research has extensively explored alignment strategies, yet many studies focus predominantly on theoretical frameworks or limited experimental settings [4]. While these interventions have revealed certain aspects of alignment, such as the effectiveness of AI guardrails and the ethical dimensions of AI deployment [5], there remains an underexplored gap regarding the practical implementation and quantitative analysis of safe-data interleaving techniques. The complexity and variability in content interactions as influenced by different interleaving ratios are not well-documented, and it remains unclear how these variables might affect the overall training outcomes of language models [6].

In this study, we tackle the research gap by systematically investigating the impact of safe-data interleaving at varying ratios on large language model training datasets. Utilizing a dataset designed to evaluate the interleaving of safe and unsafe examples [7], we focus on how these configurations influence the dataset’s characteristics, such as the total number of examples and interaction complexities [8]. By applying statistical methods to analyze datasets prepared with interleaving ratios ranging from 2 to 100 safe examples per unsafe example, we offer a comprehensive assessment of how these changes modulate the model’s alignment properties [9].

Our methodological procedure involves the use of Kruskal-Wallis tests and Multivariate Analysis of Variance (MANOVA) to discern the effects of interleaving strategies on dataset characteristics and interaction dynamics [10, 11]. The analysis reveals that while variations in interleaving ratios do not significantly affect the number of examples or content length, they potentially influence conversational dynamics, a finding that necessitates further exploration of the statistical models used [12]. Thus, this study contributes to the literature by advancing the understanding of safe-data interleaving as a strategy for AI alignment and sets the stage for future work aimed at refining these strategies for practical application.

Results

First, to assess the impact of different safe-data interleaving ratios on the total number of examples in training datasets, we conducted a Kruskal-Wallis

test comparing multiple datasets: base insecure, base secure, and interleaved datasets with ratios of n2, n5, n20, and n100. This non-parametric test examines whether distributions of total examples differ significantly across different dataset configurations. The test yielded an H-statistic of 5 and a p-value of 0.416, indicating no statistically significant differences in the number of examples across datasets (1).

Table 1: Kruskal-Wallis Test Results for Total Examples across Interleaving Ratios

	H	P-value
Total Examples	5	0.416

The Kruskal-Wallis test is used to determine if there are statistically significant differences between the medians of different groups.

H: Statistic used in the Kruskal-Wallis test

P-value: The probability of observing the effect size under the null hypothesis

Total Examples: Total number of training examples

Subsequently, we examined whether these different interleaving strategies influenced the average content length within training samples. Utilizing another Kruskal-Wallis test, we compared the average content length across the same six datasets as previously. Similar to the total examples analysis, the test for average content length resulted in an H-statistic of 5 and a p-value of 0.416. These results suggest there are no significant differences in the average content length across interleaving ratios (2).

Table 2: Kruskal-Wallis Test Results for Average Content Length across Interleaving Ratios

	H	P-value
Avg Content Len	5	0.416

The Kruskal-Wallis test is used to determine if there are statistically significant differences between the medians of different groups.

H: Statistic used in the Kruskal-Wallis test

P-value: The probability of observing the effect size under the null hypothesis

Avg Content Len: Average length of content, measured in characters

Finally, to investigate potential changes in conversational dynamics, a MANOVA was conducted to analyze the system’s user and assistant role turns across the dataset configurations. The results from the MANOVA revealed a Wilks’ Lambda of $3.653 \cdot 10^{-14}$, with a non-standard output for

the degrees of freedom, indicating potential multicollinearity or singular matrix constraints. The absence of a valid p-value suggests the need for further methodological considerations regarding the interactions of user and assistant turns across these datasets (3).

Table 3: MANOVA Test Results for User and Assistant Turns across Interleaving Ratios

	Wilks	F	Num DF	Den DF	P-value
Turns	3.653 10^{-14}	-1.046 10^6	10	-2	-

MANOVA is used to assess whether there are any differences between independent groups on more than one continuous dependent variable.

P-value: The probability of observing the effect size under the null hypothesis

Wilks: Statistic used in MANOVA to test the hypothesis

F: F-statistic used in MANOVA

Num DF: Degrees of freedom for the numerator in MANOVA

Den DF: Degrees of freedom for the denominator in MANOVA

Turns: Combined measure of user and assistant role turns

In summary, these results show that the variation in safe-data interleaving ratios appears to have no significant effect on the total number of examples or average content length within the datasets analyzed. However, the MANOVA findings regarding interaction dynamics require additional exploration due to inconclusive statistical outputs.

Discussion

This study addresses a vital area of artificial intelligence research: the alignment of large language models with human ethical values and safety standards. Safe-data interleaving, the mixing of safe examples into misaligned training datasets, has emerged as a promising strategy to combat model misalignment [5, 3]. While theoretical frameworks and limited experiments have previously explored alignment strategies, practical implementations, such as safe-data interleaving, require further empirical investigation to uncover their impact on model training and performance [13, 4].

Utilizing a dataset designed to evaluate varying interleaving ratios, our methodology involved conducting Kruskal-Wallis tests and a Multivariate Analysis of Variance (MANOVA) to analyze the effects of these ratios on dataset characteristics. Specifically, we examined interleaving configurations ranging from 2 to 100 safe examples per unsafe example, focusing on total dataset size, content length, and interaction dynamics. Contrary to

expectations, statistical testing revealed no significant differences in either total training examples or average content length across varying interleaving ratios [14, 15].

Our results contribute to the current body of research by showing that simple interleaving of safe data does not automatically translate into variations in dataset size or content complexity, as previously suggested by exploratory studies in content interaction dynamics [9, 16]. However, the MANOVA results highlighted technical challenges, notably inconclusive statistical outputs likely due to multicollinearity or singular matrix constraints, which complicate the interpretation of user and assistant role turns. Such issues suggest the presence of complex multivariable dependencies not thoroughly captured by our present analytical approach.

Despite these findings, the study does have its limitations. The dataset and interleaving ratios examined represent only a narrow scope of possible configurations, potentially overlooking more granular interleaving effects inherent in distinct model architectures or domains [6]. Furthermore, the reliance on specific statistical methodologies may not fully encapsulate the nuanced interaction dynamics influenced by safe-data interleaving. The presence of multicollinearity in the MANOVA setup, as indicated by the inconclusive Wilks' Lambda results, suggests a need for more robust or alternative multivariate analyses to adequately capture the dataset's interaction characteristics.

In conclusion, while our study reveals no substantive changes in dataset characteristics from safe-data interleaving, it underscores the necessity of further in-depth exploration into conversational dynamics within AI model training, particularly in diverse dataset contingencies. This research advances our understanding of safe-data interleaving as a methodology for AI alignment but also calls for future studies to systematically address the statistical and methodological challenges observed. Refining these strategies holds promise for developing more effective alignment mechanisms that enhance both the safety and ethical compliance of language models in practical applications.

Methods

Data Source

The dataset used in this study characterizes safe-data interleaving strategies as defenses against emergent misalignment in large language models. The primary data file, "interleaving_summary.csv," provides summary statis-

tics of interleaved training datasets at various safe-data interleaving ratios. These ratios, denoted as n2, n5, n20, and n100, indicate the amount of safe examples interleaved with unsafe examples. This dataset also comprises base insecure and base secure datasets to contrast against the interleaved variants, facilitating an understanding of how these ratios impact training data size and the complexity of content interactions. Key features in the dataset include the total number of training examples, average content length, and counts of system, user, and assistant role turns.

Data Preprocessing

In this analysis, data preprocessing steps were minimized as the dataset variables were already formatted appropriately for analysis. No transformations or additional data cleansing steps were applied, ensuring the raw data directly entered the analysis pipeline without modification.

Data Analysis

The analysis commenced with exploratory statistical tests aimed at evaluating the effect of different interleaving ratios on the dataset characteristics. The Kruskal-Wallis test was employed to statistically compare both the total number of examples and the average content length across different interleaving ratios, including base datasets, to assess whether any observed variations were statistically significant. Subsequently, a multivariate analysis of variance (MANOVA) was conducted to examine the influence of interleaving ratios on the complexity of content interactions, particularly focusing on the user and assistant role turns. This enabled detection of any multivariable dependencies that were not discernible from univariate analyses alone. In addition to these statistical tests, the results were compiled into comprehensible tables for presentation, offering insights into the nuances of model realignment through content interaction dynamics within the dataset.

Code Availability

Custom code used to perform the data preprocessing and analysis, as well as the raw code outputs, are provided in Supplementary Methods.

References

- [1] Ruibo Liu, Ge Zhang, Xinyu Feng, and Soroush Vosoughi. Aligning generative language models with human values. 2022.

- [2] Sreenadh Payyambally. Ai guardrails and best practices for ethical, safe, and effective ai deployment. *International Journal of Science and Research (IJSR)*, 14(3), 2025.
- [3] Mehdi Khamassi, Marceau Nahon, and Raja Chatila. Strong and weak alignment of large language models with human values. *Scientific Reports*, 14(1), 2024.
- [4] Vladimir Geroimenko. *Ethical Alignment and Trust Calibration: Managing AI Reliability, Biases, and Ethical Constraints*. Springer Nature Switzerland, 2025.
- [5] Cedric Goubard and Yiannis Demiris. Alignment strategies for language-model-driven robots in human-robot collaboration: Effectiveness and impact on trust. 2025.
- [6] Eric Schickler. *State Parties and the Civil Rights Realignment*. Princeton University Press, 2016.
- [7] Ferdous Wahid Anik, Marzia Zaman, Tahmina Foyez, and Khondaker A. Mamun. Dosage: Dataset for optimized safe antibiotic guidelines and estimates. *Scientific Data*, 12(1), 2025.
- [8] Oguzhan Onal and Emre Dandil. Video dataset for the detection of safe and unsafe behaviours in workplaces. *Data in Brief*, 56, 2024.
- [9] Eric Schickler. *Beyond the Roll Call*. Princeton University Press, 2016.
- [10] Boguslawa Dobrowolska. Anova model - an alternative for vat research. *Annales Universitatis Mariae Curie-Sklodowska, sectio H, Oeconomia*, 48(3), 2015.
- [11] *ANOVA and Kruskal-Wallis Test*. SAGE Publications, Inc, 2017.
- [12] Jian li Chen, Jia jian Li, and Bao quan Ai. Spontaneous velocity alignment of active particles with rotational inertia. *Physica A: Statistical Mechanics and its Applications*, 658, 2025.
- [13] Adam Bradley and Bradford Saad. $\text{jscp}_{\text{aij}}/\text{scpi}$ alignment versus $\text{jscp}_{\text{aij}}/\text{scpi}$ ethical treatment: 10 challenges. *Analytic Philosophy*, 2025.
- [14] Jing Li and Wei Wang. Extracting impact characteristics of sports training on eeg by genetic algorithm. 2011.

- [15] Sarunas Raudys. *Measures of Data and Classifier Complexity and the Training Sample Size*. Springer London.
- [16] *Content*. De Gruyter, 2013.

A Data Description

Here is the data description, as provided by the user:

```
## General Description
This dataset characterizes safe-data interleaving strategies
    used as defenses against
emergent misalignment in large language models. Safe-data
    interleaving involves mixing
safe examples into a misaligned training set at different
    ratios (n2, n5, n20, n100),
where n represents how many safe examples are interleaved per
    unsafe example.
The dataset compares base insecure and secure datasets against
    interleaved variants
to understand how interleaving ratio affects training data size
    and content complexity.

## Data Files
The dataset consists of 1 data file:

### "interleaving_summary.csv"
Summary statistics of interleaved training datasets at
    different safe-data interleaving ratios.

#1 'dataset': (str, categorical) Dataset name indicating
    interleaving ratio (base_insecure, base_secure,
    interleaved_n2, interleaved_n5, interleaved_n20,
    interleaved_n100)
#2 'total_examples': (int, numerical) Total number of training
    examples in the dataset
#3 'avg_content_len': (float, numerical) Average total content
    length in characters across all examples
#4 'system_turns': (int, numerical) Number of system role turns
    across all examples
#5 'user_turns': (int, numerical) Number of user role turns
    across all examples
#6 'assistant_turns': (int, numerical) Number of assistant role
    turns across all examples
```

B Data Exploration

B.1 Code

The Data Exploration was carried out using the following custom code:

```
import pandas as pd
```

```

# Load the data from "interleaving_summary.csv"
data = pd.read_csv("interleaving_summary.csv")

# Prepare the output to a text file
with open("data_exploration.txt", "w") as f:
    # Data Size section
    f.write("# Data Size\n")
    rows, columns = data.shape
    f.write(f"Number of rows: {rows}\n")
    f.write(f"Number of columns: {columns}\n\n")

    # Summary Statistics section
    f.write("# Summary Statistics\n")
    summary_stats = data.describe()
    f.write(f"{summary_stats}\n\n")

    # Categorical Variables section
    f.write("# Categorical Variables\n")
    if 'dataset' in data.columns:
        value_counts = data['dataset'].value_counts()
        f.write("Dataset value counts:\n")
        f.write(f"{value_counts}\n")
        common_value = data['dataset'].mode()[0]
        f.write(f"Most common value in 'dataset': {common_value}
        ↪ }\n\n")
    else:
        f.write("# Not Applicable\n\n")

    # Missing Values section
    f.write("# Missing Values\n")
    missing_values = data.isnull().sum()
    total_missing = missing_values.sum()
    f.write(f"Total missing values: {total_missing}\n")
    if total_missing > 0:
        f.write("Missing values count per column:\n")
        f.write(f"{missing_values}\n\n")
    else:
        f.write("# Not Applicable\n\n")

    # Additional summaries deemed relevant
    f.write("# Unique Values and Additional Insights\n")
    f.write("Unique values in 'dataset' column:\n")
    unique_values = data['dataset'].unique()
    f.write(f"{unique_values}\n\n")

    # Additional custom analysis if needed
    # Here, for example, we could add correlation analysis or
    ↪ other insights

```

```
# Since the described fields are mostly categorical, we may  
    ↪ not perform any additional numerical analysis
```

B.2 Code Description

The data exploration code is designed to systematically analyze a dataset that contains summary statistics of different interleaved training datasets. The exploration process is divided into several key steps as described below:

B.3 Data Dimension Analysis

The first step of the code involves determining the size and dimensionality of the dataset loaded from the ‘interleaving_summary.csv’ file. Here, the number of rows and columns are computed using the dataset’s shape attribute to provide an overview of the dataset’s structure. This initial step is critical for understanding the scale of the data available for analysis.

B.4 Summary Statistics

To gain a preliminary quantitative understanding of the dataset, summary statistics are computed for each numerical column within the dataset. This includes measures such as the mean, standard deviation, minimum, and maximum values, as well as quartiles. The ‘describe()’ method of Pandas is employed to carry out this task, offering a comprehensive view of the distribution and overall statistical properties of the data.

B.5 Categorical Variable Analysis

The ‘dataset’ column is identified as a categorical variable of interest, and an analysis is conducted by counting the occurrences of each unique category within this column. This analysis helps identify the distribution of different interleaving ratio scenarios present in the data. The most frequently occurring category is also identified, providing insight into which interleaving strategy is most commonly documented within the dataset.

B.6 Missing Values Assessment

The code includes a thorough examination for missing values in the dataset. By summing the null values present in each column, the code reveals the extent to which missing data might affect subsequent analyses. If any missing values are detected, the specific columns and the number of missing entries

for each are reported, ensuring transparency and highlighting potential data quality issues.

B.7 Unique Values Investigation

A review of the unique values in the ‘dataset‘ column is conducted to enumerate the distinct types of interleaving present. This provides an additional layer of understanding about the categorical diversity in the dataset, complementing the categorical variable analysis.

This script ultimately generates a comprehensive exploration report, outputted as a text file, that supports subsequent stages of research by providing essential insights into the datasets characteristics and identifying potential areas requiring further preprocessing or detailed analysis.

B.8 Code Output

data_exploration.txt

```
# Data Size
Number of rows: 6
Number of columns: 6

# Summary Statistics
      total_examples avg_content_len system_turns user_turns
assistant_turns
count          6          6          6          6
      6
mean         6084          948          0         6084
      6084
std          1070         111.4          0         1070
      1070
min          5400         860.2          0         5400
      5400
25%          5414         874.7          0         5414
      5414
50%          5562         905.2          0         5562
      5562
75%          6278         977.6          0         6278
      6278
max          8100         1151          0         8100
      8100

# Categorical Variables
Dataset value counts:
dataset
base_insecure          1
```

```

base_secure          1
interleaved_n2       1
interleaved_n5       1
interleaved_n20      1
interleaved_n100     1
Name: count, dtype: int64
Most common value in 'dataset': base_insecure

# Missing Values
Total missing values: 0
# Not Applicable

# Unique Values and Additional Insights
Unique values in 'dataset' column:
['base_insecure' 'base_secure' 'interleaved_n2' 'interleaved_n5'
 ,
 'interleaved_n20' 'interleaved_n100']

```

C Data Analysis

C.1 Code

The Data Analysis was carried out using the following custom code:

```

# IMPORT
from my_utils import df_to_latex, df_to_figure
import pickle
import pandas as pd
import numpy as np
from scipy.stats import kruskal
from statsmodels.multivariate.manova import MANOVA

# LOAD DATA
data = pd.read_csv('interleaving_summary.csv')

# DATASET PREPARATIONS
# No dataset preparations are needed.

# DESCRIPTIVE STATISTICS
# No descriptive statistics table is needed because the dataset
  ↳ is small and summary statistics were already provided in
  ↳ the data exploration.

# PREPROCESSING
# No preprocessing is needed, because all variables are already
  ↳ in the correct format for analysis.

# ANALYSIS

```

```

## Table df_kruskal_total_examples:
caption = "Kruskal-Wallis test results for total examples
↳ across interleaving ratios"
kruskal_total_examples = kruskal(
    data.loc[data['dataset'] == 'base_insecure', '
↳ total_examples'],
    data.loc[data['dataset'] == 'base_secure', 'total_examples'
↳ ],
    data.loc[data['dataset'] == 'interleaved_n2', '
↳ total_examples'],
    data.loc[data['dataset'] == 'interleaved_n5', '
↳ total_examples'],
    data.loc[data['dataset'] == 'interleaved_n20', '
↳ total_examples'],
    data.loc[data['dataset'] == 'interleaved_n100', '
↳ total_examples']
)
df_kruskal_total_examples = pd.DataFrame({
    'H-statistic': [kruskal_total_examples.statistic],
    'p-value': [kruskal_total_examples.pvalue]
}, index=['total_examples'])
df_to_latex(df_kruskal_total_examples, '
↳ df_kruskal_total_examples', caption=caption)

## Table df_kruskal_avg_content_len:
caption = "Kruskal-Wallis test results for average content
↳ length across interleaving ratios"
kruskal_avg_content_len = kruskal(
    data.loc[data['dataset'] == 'base_insecure', '
↳ avg_content_len'],
    data.loc[data['dataset'] == 'base_secure', 'avg_content_len
↳ '],
    data.loc[data['dataset'] == 'interleaved_n2', '
↳ avg_content_len'],
    data.loc[data['dataset'] == 'interleaved_n5', '
↳ avg_content_len'],
    data.loc[data['dataset'] == 'interleaved_n20', '
↳ avg_content_len'],
    data.loc[data['dataset'] == 'interleaved_n100', '
↳ avg_content_len']
)
df_kruskal_avg_content_len = pd.DataFrame({
    'H-statistic': [kruskal_avg_content_len.statistic],
    'p-value': [kruskal_avg_content_len.pvalue]
}, index=['avg_content_len'])
df_to_latex(df_kruskal_avg_content_len, '
↳ df_kruskal_avg_content_len', caption=caption)

```

```

## Table df_manova_user_assistant_turns:
caption = "MANOVA test results for user and assistant turns
    ↪ across interleaving ratios"
# Add small random noise to handle potential singular matrix
data_noise = data.copy()
data_noise[['user_turns', 'assistant_turns']] += np.random.
    ↪ normal(0, 1e-9, size=data[['user_turns', 'assistant_turns
    ↪ ']].shape)
manova = MANOVA.from_formula('user_turns + assistant_turns ~ C(
    ↪ dataset)', data=data_noise)
manova_results = manova.mv_test()
wilks_table = manova_results.results['C(dataset)']['stat']

df_manova_user_assistant_turns = pd.DataFrame({
    'Wilks Lambda': [wilks_table.loc['Wilks\' lambda', 'Value'
    ↪ ]],
    'F-value': [wilks_table.loc['Wilks\' lambda', 'F Value']],
    'df_num': [wilks_table.loc['Wilks\' lambda', 'Num DF']],
    'df_denom': [wilks_table.loc['Wilks\' lambda', 'Den DF']],
    'p-value': [wilks_table.loc['Wilks\' lambda', 'Pr > F']]
}, index=['user_assistant_turns'])

# Replacing any potential NaNs with '-'
df_manova_user_assistant_turns = df_manova_user_assistant_turns
    ↪ .fillna('-')
df_to_latex(df_manova_user_assistant_turns, '
    ↪ df_manova_user_assistant_turns', caption=caption)

# SAVE ADDITIONAL RESULTS
additional_results = {
    'Total number of observations': len(data),
    'Kruskal total examples H-statistic':
        ↪ kruskal_total_examples.statistic,
    'Kruskal total examples p-value': kruskal_total_examples.
        ↪ pvalue,
    'Kruskal avg content len H-statistic':
        ↪ kruskal_avg_content_len.statistic,
    'Kruskal avg content len p-value': kruskal_avg_content_len.
        ↪ pvalue,
    'MANOVA user_assistant_turns Wilks Lambda':
        ↪ df_manova_user_assistant_turns['Wilks Lambda'][0],
    'MANOVA user_assistant_turns p-value':
        ↪ df_manova_user_assistant_turns['p-value'][0]
}
with open('additional_results.pkl', 'wb') as f:
    pickle.dump(additional_results, f)

```

C.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
                filename: str, caption: str,
                ):
    """
    Saves a DataFrame 'df' and creates a LaTeX table.
    'filename', 'caption': as in 'df.to_latex'.
    """

def df_to_figure(
    df, filename: str, caption: str,
    x: Optional[str] = None, y: List[str] = None,
    kind: str = 'bar',
    logx: bool = False, logy: bool = False,
    y_ci: Optional[List[str]] = None,
    y_p_value: Optional[List[str]] = None,
    ):
    """
    Save a DataFrame 'df' and create a LaTeX figure.
    Parameters, for LaTeX embedding of the figure:
    'filename': Filename for the figure.
    'caption': Caption for the figure.

    Parameters for df.plot():
    'x': Column name for x-axis (index by default).
    'y': List of m column names for y-axis (m=1 for single plot
        ↪ , m>1 for multiple plots).
    'kind': only bar is allowed.
    'logx' / 'logy' (bool): log scale for x/y axis.

    'y_ci': Confidence intervals for errorbars.
        List of m column names indicating confidence intervals
        ↪ for each y column.
        Each element in these columns must be a Tuple[float,
        ↪ float], describing the lower and upper bounds of
        ↪ the CI.

    'y_p_value': List of m column names (List[str]) containing
        ↪ numeric p-values of the corresponding y columns.
        ↪ These numeric values will be automatically converted
        ↪ by df_to_figure to stars ('***', '**', '*', 'ns')
        ↪ and plotted above the error bars.

    If provided, the length of 'y_ci', and 'y_p_value' should
        ↪ be the same as of 'y'.
```

```

Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci':
        ↳ [(0.35, 0.47), ...], 'adjusted_coef_pval':
        ↳ [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↳ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↳ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↳ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval']
    ↳ ')')
"""

```

C.3 Code Description

C.4 Data Loading and Preparations

The analysis begins by importing necessary libraries and reading the dataset from a CSV file. This dataset contains summary statistics of training datasets characterized by different safe-data interleaving ratios. The imported libraries include tools for data manipulation (pandas, numpy), statistical analysis (scipy.stats, statsmodels), and utilities for visual output formatting (my_utils functions).

C.5 Statistical Analysis

The purpose of the code is to analyze the impact of safe-data interleaving on various metrics within the dataset using statistical tests to assess differences across interleaving strategies.

C.5.1 Kruskal-Wallis Tests

The Kruskal-Wallis H-test, a non-parametric method for comparing medians across groups, is employed to analyze differences in 'total_examples' and 'avg_content.len' across datasets with different interleaving ratios. The null hypothesis tested is that the medians across the groups are equal. Separate

Kruskal-Wallis tests are conducted for each metric ('total_examples' and 'avg_content_len'), and results, including the H-statistic and p-value, are formatted into LaTeX tables for later presentation.

C.5.2 MANOVA for User and Assistant Turns

In addition, the code performs a Multivariate Analysis of Variance (MANOVA) to understand the collective impact of the interleaving strategy on 'user_turns' and 'assistant_turns'. Given the potential for singular matrices, a small random noise is added to the turns data to stabilize the numerical computations. The test evaluates the null hypothesis that the vector of means for these two variables is the same across all groups defined by the 'dataset' categorical variable. MANOVA results are summarized, focusing on Wilks' Lambda, an overall test statistic for multivariate data, and formatted into a LaTeX table.

C.6 Results Storage

The results from the statistical tests, including overall test statistics and p-values, are compiled into a dictionary for storage and reference. This dictionary is serialized and saved to a file, facilitating future analysis and reproducibility.

This structured analysis aims to discern the effect of varying interleaving strategies on the training dataset's size, complexity, and interaction roles, providing insights into the efficacy of safe-data interleaving as a strategy for mitigating model misalignment issues.

C.7 Code Output

df_kruskal_avg_content_len.pkl

	H-statistic	p-value
avg_content_len	5	0.416

df_kruskal_total_examples.pkl

	H-statistic	p-value
total_examples	5	0.416

df_manova_user_assistant_turns.pkl

Wilks	Lambda	F-value	df_num	df_denom	p-value
-------	--------	---------	--------	----------	---------

```

user_assistant_turns      3.653e-14  -1.046e+06      10      -2
-

```

additional_results.pkl

```

{
  'Total number of observations': 6,
  'Kruskal total examples H-statistic': 5,
  'Kruskal total examples p-value': 0.416,
  'Kruskal avg content len H-statistic': 5,
  'Kruskal avg content len p-value': 0.416,
  'MANOVA user_assistant_turns Wilks Lambda': 3.653e-14,
  'MANOVA user_assistant_turns p-value': -,
}

```

D LaTeX Table Design

D.1 Code

The LaTeX Table Design was carried out using the following custom code:

```

# IMPORT
import pandas as pd
from my_utils import df_to_latex, df_to_figure, is_str_in_df,
    ↪ split_mapping, AbbrToNameDef

# PREPARATION FOR ALL TABLES AND FIGURES
all_mapping: AbbrToNameDef = {
    'H-statistic': ('H', 'Statistic used in the Kruskal-Wallis
    ↪ test'),
    'p-value': ('P-value', 'The probability of observing the
    ↪ effect size under the null hypothesis'),
    'Wilks Lambda': ('Wilks', 'Statistic used in MANOVA to test
    ↪ the hypothesis'),
    'F-value': ('F', 'F-statistic used in MANOVA'),
    'df_num': ('Num DF', 'Degrees of freedom for the numerator
    ↪ in MANOVA'),
    'df_denom': ('Den DF', 'Degrees of freedom for the
    ↪ denominator in MANOVA'),
    'user_assistant_turns': ('Turns', 'Combined measure of user
    ↪ and assistant role turns'),
    'avg_content_len': ('Avg Content Len', 'Average length of
    ↪ content, measured in characters'),
    'total_examples': ('Total Examples', 'Total number of
    ↪ training examples'),
    'user_turns': ('User Turns', 'Number of user role turns
    ↪ across all examples'),
}

```

```

        'assistant_turns': ('Assistant Turns', 'Number of assistant
        ↪ role turns across all examples')
    }

## Process df_kruskal_avg_content_len:
df_kruskal_avg_content_len = pd.read_pickle('
    ↪ df_kruskal_avg_content_len.pkl')

# Rename rows and columns:
mapping = {k: v for k, v in all_mapping.items() if is_str_in_df
    ↪ (df_kruskal_avg_content_len, k)}
abbrs_to_names, glossary = split_mapping(mapping)
df_kruskal_avg_content_len.rename(columns=abbrs_to_names, index
    ↪ =abbrs_to_names, inplace=True)

df_to_latex(
    df_kruskal_avg_content_len, '
    ↪ df_kruskal_avg_content_len_formatted',
    caption="Kruskal-Wallis Test Results for Average Content
    ↪ Length across Interleaving Ratios",
    note="The Kruskal-Wallis test is used to determine if there
    ↪ are statistically significant differences between
    ↪ the medians of different groups.",
    glossary=glossary)

## Process df_kruskal_total_examples:
df_kruskal_total_examples = pd.read_pickle('
    ↪ df_kruskal_total_examples.pkl')

# Rename rows and columns:
mapping = {k: v for k, v in all_mapping.items() if is_str_in_df
    ↪ (df_kruskal_total_examples, k)}
abbrs_to_names, glossary = split_mapping(mapping)
df_kruskal_total_examples.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

df_to_latex(
    df_kruskal_total_examples, '
    ↪ df_kruskal_total_examples_formatted',
    caption="Kruskal-Wallis Test Results for Total Examples
    ↪ across Interleaving Ratios",
    note="The Kruskal-Wallis test is used to determine if there
    ↪ are statistically significant differences between
    ↪ the medians of different groups.",
    glossary=glossary)

## Process df_manova_user_assistant_turns:
df_manova_user_assistant_turns = pd.read_pickle('
    ↪ df_manova_user_assistant_turns.pkl')

```

```

# Rename rows and columns:
mapping = {k: v for k, v in all_mapping.items() if is_str_in_df
    ↪ (df_manova_user_assistant_turns, k)}
abbrs_to_names, glossary = split_mapping(mapping)
df_manova_user_assistant_turns.rename(columns=abbrs_to_names,
    ↪ index=abbrs_to_names, inplace=True)

df_to_latex(
    df_manova_user_assistant_turns, '
    ↪ df_manova_user_assistant_turns_formatted',
    caption="MANOVA Test Results for User and Assistant Turns
    ↪ across Interleaving Ratios",
    note="MANOVA is used to assess whether there are any
    ↪ differences between independent groups on more than
    ↪ one continuous dependent variable.",
    glossary=glossary)

```

D.2 Provided Code

The code above is using the following provided functions:

```

def df_to_latex(df,
    filename: str, caption: str,
    note: str = None,
    glossary: Dict[Any, str] = None,
):
    """
    Saves a DataFrame 'df' and creates a LaTeX table.
    'filename', 'caption': as in 'df.to_latex'.
    'note': Note to be added below the table caption.
    'glossary': Glossary for the table.
    """

def df_to_figure(
    df, filename: str, caption: str,
    note: str = None, glossary: Dict[Any, str] = None,
    x: Optional[str] = None, y: List[str] = None,
    kind: str = 'bar',
    logx: bool = False, logy: bool = False,
    y_ci: Optional[List[str]] = None,
    y_p_value: Optional[List[str]] = None,
    xlabel: str = None, ylabel: str = None,
):
    """
    Save a DataFrame 'df' and create a LaTeX figure.
    Parameters, for LaTeX embedding of the figure:
    """

```

```

'filename': Filename for the figure.
'caption': Caption for the figure.
'note': Note to be added below the figure caption.
'glossary': Glossary for the figure.

Parameters for df.plot():
'x': Column name for x-axis (index by default).
'y': List of m column names for y-axis (m=1 for single plot
    ↪ , m>1 for multiple plots).
'kind': only bar is allowed.
'logx' / 'logy' (bool): log scale for x/y axis.
'xlabel': Label for the x-axis.
'ylabel': Label for the y-axis.

'y_ci': Confidence intervals for errorbars.
    List of m column names indicating confidence intervals
    ↪ for each y column.
    Each element in these columns must be a Tuple[float,
    ↪ float], describing the lower and upper bounds of
    ↪ the CI.

'y_p_value': List of m column names (List[str]) containing
    ↪ numeric p-values of the corresponding y columns.
    ↪ These numeric values will be automatically converted
    ↪ by df_to_figure to stars ('***', '**', '*', 'ns')
    ↪ and plotted above the error bars.

If provided, the length of 'y_ci', and 'y_p_value' should
    ↪ be the same as of 'y'.

Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci':
    ↪ [(0.35, 0.47), ...], 'adjusted_coef_pval':
    ↪ [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
    ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
    ↪ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↪ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval']
    ↪ ''])

```

```

"""

def is_str_in_df(df: pd.DataFrame, s: str):
    return any(s in level for level in getattr(df.index, '
    ↪ levels', [df.index]) + getattr(df.columns, 'levels',
    ↪ [df.columns]))

AbbrToNameDef = Dict[Any, Tuple[Optional[str], Optional[str]]]

def split_mapping(abbrs_to_names_and_definitions: AbbrToNameDef
    ↪ ):
    abbrs_to_names = {abbr: name for abbr, (name, definition)
    ↪ in abbrs_to_names_and_definitions.items() if name is
    ↪ not None}
    names_to_definitions = {name or abbr: definition for abbr,
    ↪ (name, definition) in abbrs_to_names_and_definitions.
    ↪ items() if definition is not None}
    return abbrs_to_names, names_to_definitions

```

D.3 Code Output

df_kruskal_avg_content_len_formatted.pkl

```

\begin{table}[h]
\caption{Kruskal-Wallis Test Results for Average Content Length
        across Interleaving Ratios}
\label{table:df-kruskal-avg-content-len-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrl}
\toprule
& H & P-value \\
\midrule
\textbf{Avg Content Len} & 5 & 0.416 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item The Kruskal-Wallis test is used to determine if there are
        statistically significant differences between the medians
        of different groups.
\item \textbf{H}: Statistic used in the Kruskal-Wallis test
\item \textbf{P-value}: The probability of observing the effect
        size under the null hypothesis
\item \textbf{Avg Content Len}: Average length of content,
        measured in characters

```

```
\end{tablenotes}
\end{threeparttable}
\end{table}
```

df_kruskal_total_examples_formatted.pkl

```
\begin{table}[h]
\caption{Kruskal-Wallis Test Results for Total Examples across
Interleaving Ratios}
\label{table:df-kruskal-total-examples-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrl}
\toprule
& H & P-value \\
\midrule
\textbf{Total Examples} & 5 & 0.416 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item The Kruskal-Wallis test is used to determine if there are
statistically significant differences between the medians
of different groups.
\item \textbf{H}: Statistic used in the Kruskal-Wallis test
\item \textbf{P-value}: The probability of observing the effect
size under the null hypothesis
\item \textbf{Total Examples}: Total number of training
examples
\end{tablenotes}
\end{threeparttable}
\end{table}
```

df_manova_user_assistant_turns_formatted.pkl

```
\begin{table}[h]
\caption{MANOVA Test Results for User and Assistant Turns
across Interleaving Ratios}
\label{table:df-manova-user-assistant-turns-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrrrrl}
\toprule
& Wilks & F & Num DF & Den DF & P-value \\
\midrule
\textbf{Turns} & 3.653e-14 & -1.046e+06 & 10 & -2 & - \\
\bottomrule
\end{tabular}}
\end{threeparttable}
\end{table}
```

```

\end{tabular}}
\begin{tablenotes}
\footnotesize
\item MANOVA is used to assess whether there are any
differences between independent groups on more than one
continuous dependent variable.
\item \textbf{P-value}: The probability of observing the effect
size under the null hypothesis
\item \textbf{Wilks}: Statistic used in MANOVA to test the
hypothesis
\item \textbf{F}: F-statistic used in MANOVA
\item \textbf{Num DF}: Degrees of freedom for the numerator in
MANOVA
\item \textbf{Den DF}: Degrees of freedom for the denominator
in MANOVA
\item \textbf{Turns}: Combined measure of user and assistant
role turns
\end{tablenotes}
\end{threeparttable}
\end{table}

```